

Milestone 2: GridBrief AI Tech Stack

Learning goals

- Break the application into key architectural layers.
- Choose a realistic implementation stack for a short project.
- Propose an “ideal” stack and explain trade-offs (cost, complexity, scalability).

AI Tech Stack Table

Yellow = implemented today. Green = ideal future component.

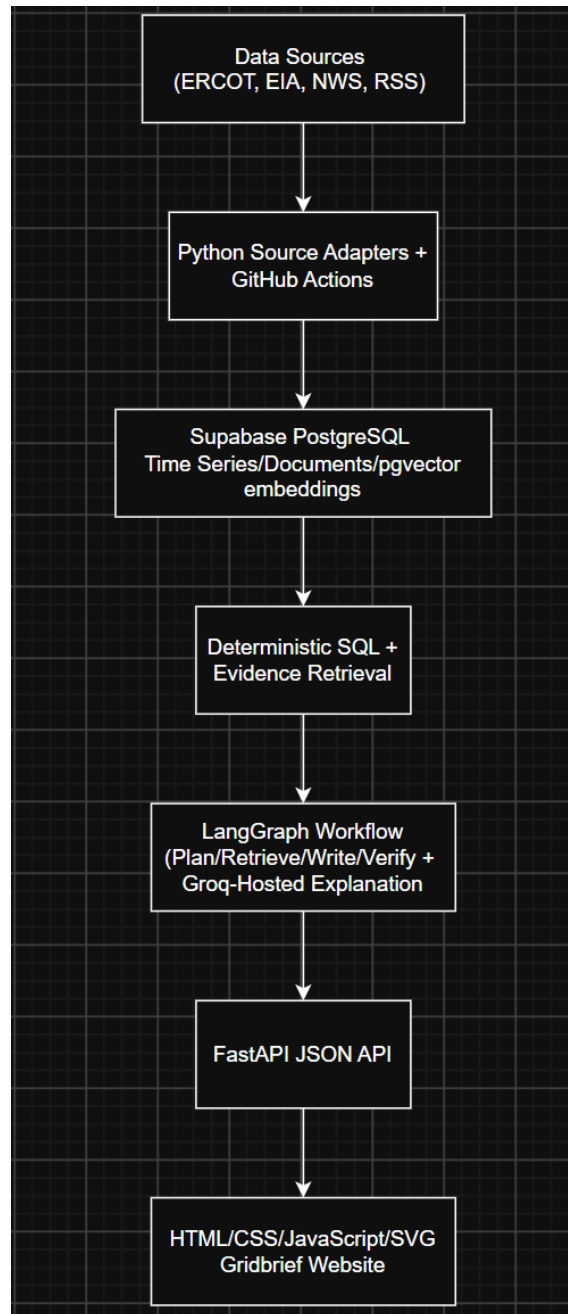
Layer (back → front)	Really basic (local/simple)	Freemium (hosted, good free tier)	Google (paid)	AWS (paid)	Microsoft (paid)	Others (paid)
Data layer & vector store	CSV, JSON, Excel	SQLite Supabase Postgres + pgvector [IMPLEMENTED]	BigQuery [IDEAL]	Amazon RDS	Azure SQL Database	Snowflake
Model training & MLOps	Jupyter Notebook, Visual Studio Code	Google Colab, Weights & Biases GitHub Actions + pytest [IMPLEMENTED]	Vertex AI [IDEAL]	Amazon SageMaker	Azure Machine Learning	Databricks
Coding / IDE & agents	Simple text editor, terminal scripts [IMPLEMENTED]	Cursor Visual Studio Code, GitHub + LangGraph, Claude Code, LangSmith [IMPLEMENTED]	—	—	GitHub Copilot	Claude Code, Codeium Cody, LangChain, Crew, N8N
Gen AI models	Ollama + local Llama/ Mistral/Gemma	OpenAI free tier, Gemini free tier, Claude web free tier Groq-hosted generative LLM BGE-base-en-v1.5 embeddings [IMPLEMENTED]	Gemini API [IDEAL] FAILOVER	Amazon Bedrock	Azure OpenAI	Anthropic Claude API, Mistral API Managed reranker [IDEAL]
Frontend UI & dev tools	PowerPoint, Excel	Streamlit, React, Next.js, Vercel free tier	Firebase Hosting	AWS Amplify	Azure Static Web Apps	Figma, Framer, Webflow

	HTML, CSS, JavaScript, SVG [IMPLEMENTED]	API: FastAPI + Uvicorn [IMPLEMENTED]				Next.js + PostHog [IDEAL]
Hosting / runtime	Python HTTP server Docker + localhost [IMPLEMENTED]	Vercel, Netlify, Fly.io, Cloudflare Pages (free) Render + Supabase + GitHub Actions [IMPLEMENTED]	Cloud Run [IDEAL]	AWS Lambda / ECS	Azure App Service	Cloudflare Workers, Heroku, Kubernetes
Quality, security & observability	Ruff + pytest [IMPLEMENTED]	Citations, rate limits, caching [IMPLEMENTED]	Cloud Monitoring	CloudWatch	Application Insights	Sentry + OpenTelemetry [IDEAL]

1. What stack did you implement? (Yellow Highlights + IMPLEMENTED)

We implemented a Python/FastAPI application with native HTML/CSS/JavaScript, LangGraph agent orchestration, Groq inference, deterministic SQL analytics, BGE embeddings, pgvector retrieval in Supabase PostgreSQL, GitHub Actions automation, Docker and Render hosting. Yellow highlights show the implemented components.

Data Sources (ERCOT, EIA, NWS, RSS) → Python Source Adapters + GitHub Actions → Supabase PostgreSQL Time Series/Documents/pgvector embeddings → Deterministic SQL + Evidence Retrieval → LangGraph Workflow (Plan/Retrieve/Write/Verify) + Groq-Hosted Explanation → FastAPI JSON API → HTML/CSS/JavaScript/SVG Gridbrief Website



Link to Draw.io: https://drive.google.com/file/d/1PUM3IHb7uh6iQmCePvExNvYrTTLUuj_q/view?usp=sharing

2. What is your ideal stack? (Green Highlights + IDEAL)

The ideal production stack evolves on the current stack by adding Cloud Run autoscaling, BigQuery for long-horizon analytics, managed observability with Sentry/OpenTelemetry or LangSmith, a reranker, and multi-provider model failover. Green highlights show these proposed components.

3. Why did you pick these ideal stack components?

They improve reliability, debugging, retrieval quality, and burst scalability. The trade-off is higher recurring cost, more vendor coupling, and greater operational complexity, so they are appropriate after usage justifies them, not for the current free public version hosted.